

5.2.49

RAVULA SHASHANK REDDY - EE25BTECH11047

October 12, 2025

Question

Consider the basis

$$\{\mathbf{u}_1, \mathbf{u}_2, \mathbf{u}_3\} \text{ of } \mathbb{R}^3, \text{ where } \mathbf{u}_1 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \mathbf{u}_2 = \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix}, \mathbf{u}_3 = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}.$$

Let $\{\mathbf{f}_1, \mathbf{f}_2, \mathbf{f}_3\}$ be the dual basis and $f(a, b, c) = a + b + c$. If

$$\mathbf{f} = \alpha_1 \mathbf{f}_1 + \alpha_2 \mathbf{f}_2 + \alpha_3 \mathbf{f}_3,$$

find $(\alpha_1 \quad \alpha_2 \quad \alpha_3)$.

Solution

The dual basis satisfies :

$$\mathbf{f}_i(\mathbf{u}_j) = \delta_{ij} = \begin{cases} 1, & i = j \\ 0, & i \neq j \end{cases}. \quad (1)$$

$$\mathbf{U} = (\mathbf{u}_1 \ \mathbf{u}_2 \ \mathbf{u}_3) = \begin{pmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{pmatrix}, \quad (2)$$

$$\mathbf{F} = \begin{pmatrix} \mathbf{f}_1 \\ \mathbf{f}_2 \\ \mathbf{f}_3 \end{pmatrix}. \quad (3)$$

Then the dual basis condition becomes

$$\mathbf{F} \mathbf{U} = \mathbf{I}_3. \quad (4)$$

$$\mathbf{F}^{-1} = \mathbf{U} \quad (5)$$

$$\mathbf{f} = \alpha_1 \mathbf{f}_1 + \alpha_2 \mathbf{f}_2 + \alpha_3 \mathbf{f}_3 \implies \mathbf{f} = \mathbf{a}^T \mathbf{F}, \quad (6)$$

$$\text{where } \mathbf{a} = \begin{pmatrix} \alpha_1 \\ \alpha_2 \\ \alpha_3 \end{pmatrix} \quad \mathbf{f} = \begin{pmatrix} 1 & 1 & 1 \end{pmatrix} \quad (7)$$

$$\mathbf{a}^T \mathbf{F} = \mathbf{f} \implies \mathbf{a}^T = \mathbf{f} \mathbf{F}^{-1} = \mathbf{f} \mathbf{U}, \quad (8)$$

$$\mathbf{a}^T = \begin{pmatrix} 1 & 1 & 1 \end{pmatrix} \begin{pmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 2 & 3 \end{pmatrix}. \quad (9)$$

$$\begin{pmatrix} \alpha_1 \\ \alpha_2 \\ \alpha_3 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix} \quad (10)$$

C Code

```
#include <stdio.h>

int main(void) {
    // Basis matrix U: columns are u1,u2,u3 but stored row-major
    // U[row][col]
    double U[3][3] = {
        {1.0, 1.0, 1.0},
        {0.0, 1.0, 1.0},
        {0.0, 0.0, 1.0}
    };

    // f as row vector (f = (1,1,1))
    double f[3] = {1.0, 1.0, 1.0};

    // alpha will store the row vector alpha = f * U
    double alpha[3] = {0.0, 0.0, 0.0};

    // Compute alpha_j = sum_k f_k * U[k][j]
```

C Code

```
for (int j = 0; j < 3; ++j) {
    double sum = 0.0;
    for (int k = 0; k < 3; ++k) {
        sum += f[k] * U[k][j];
    }
    alpha[j] = sum;
}

// Print result
printf(alpha = ());
for (int j = 0; j < 3; ++j) {
    printf(%.0f, alpha[j]); // integer values expected, print
                             without decimals
    if (j < 2) printf(, );
}
printf()\n);

return 0;
}
```

```
import numpy as np

# Given basis matrix U
U = np.array([
    [1, 1, 1],
    [0, 1, 1],
    [0, 0, 1]
], dtype=float)

# Given linear functional f(a,b,c) = a + b + c
f = np.array([1, 1, 1], dtype=float)

# Compute  $\alpha^T = f * U$ 
alpha = f @ U # matrix multiplication

print(alpha =, alpha)
```

Python Shared

```
import ctypes
import numpy as np

# Load the shared library
lib = ctypes.CDLL(./libalpha.so)

# Define argument and return types
lib.compute_alpha.argtypes = [
    (ctypes.c_double * 3 * 3), # 3x3 matrix
    (ctypes.c_double * 3), # f vector
    (ctypes.c_double * 3) # output alpha
]

# Prepare data
U = np.array([
    [1.0, 1.0, 1.0],
    [0.0, 1.0, 1.0],
    [0.0, 0.0, 1.0]
```

```
], dtype=np.double)

f = np.array([1.0, 1.0, 1.0], dtype=np.double)
alpha = np.zeros(3, dtype=np.double)

# Convert numpy arrays to ctypes
U_ctypes = (ctypes.c_double * 3 * 3)(*map(lambda row: (ctypes.c_double * 3)(*row), U))
f_ctypes = (ctypes.c_double * 3)(*f)
alpha_ctypes = (ctypes.c_double * 3)(*alpha)

# Call C function
lib.compute_alpha(U_ctypes, f_ctypes, alpha_ctypes)

# Convert result back to numpy array
alpha_result = np.array(list(alpha_ctypes))

print(alpha =, alpha_result)
```